

Data Vis practice8

```
# Load necessary libraries
library(shiny)
library(ggplot2)
library(dplyr)
library(plotly)
library(httr)
library(jsonlite)
library(fmsb)
library(scales)
library(tidyr)
library(readr)

# --- Data Preparation ---

# Load the music data (make sure the CSV file path is correct)
music_data <- read_csv("Spotify_Youtube .csv")

music_data <- music_data %>%
  mutate(
    YouTube_Revenue = ifelse(Views <= 10000, Views * 0.06,
    Views * 0.12),
    Spotify_Revenue = Stream * (0.04 / 10),
    Total_Revenue = YouTube_Revenue + Spotify_Revenue
  )

# Define the list of Korean artists
korean_artists <- c("BTS", "EXO", "BLACKPINK", "TWICE", "Stray Kids",
                    "Red Velvet", "NCT", "SEVENTEEN", "BIGBANG",
                    "SHINee", "(G)I-DLE", "ATEEZ", "TOMORROW X TOGETHER")
```

```

W X TOGETHER",
                                "ENHYPEN", "IVE", "LE SSERAFIM", "NewJe
ans",
                                "TAEYANG", "RM", "Jimin", "j-hope", "I
U",
                                "Girls' Generation", "JAY PARK", "ITZ
Y")

# --- Shiny UI ---

ui <- fluidPage(
  titlePanel("Music Data Analysis Dashboard"),
  navbarPage("",
    tabPanel("Artist Revenue Distribution",
      sidebarLayout(
        sidebarPanel(
          textInput("api_key", "Enter your
OpenAI API Key:", value = "", placeholder = "Your OpenAI AP
I Key"),
          selectInput("xaxis",
                        "Select Feature for X
-axis:",
                        choices = names(music
_data)[!(names(music_data) %in% c("Artist", "Total_Revenu
e"))],
                        selected = "Energy"),
          sliderInput("globalCount",
                      "Select Top Percentag
e of Global Artists:",
                      min = 1.5,
                      max = 100,
                      value = 1.5),
          actionButton("analyze", "Analyze
with GPT")
        ),
        mainPanel(
          plotlyOutput("distributionPlot"),
          textOutput("gpt_analysis")
        )
      )
    )
  )

```

```

    )
  )
),
  tabPanel("Song Radar Chart Comparison",
    sidebarLayout(
      sidebarPanel(
        selectInput("selected_song", "Select a song:",
                      choices = unique(music_data$Track))
      ),
      mainPanel(
        h3("Main Radar Chart (Selected Song)"),

        plotOutput("mainRadarChart"),
        h3("Radar Chart (Top 3 Similar Songs)"),

        plotOutput("similarRadarChart"),
        h4("Artist:"),
        textOutput("artistName"),
        h3("Top 3 Similar Songs:"),
        tableOutput("similarSongs")
      )
    )
  )
)

# --- Shiny Server ---

server <- function(input, output, session) {

  # --- Tab 1: Artist Revenue Distribution ---

  # Reactive expression to calculate x-axis limits
  overall_x_limits <- reactive({
    x_values <- music_data[[input$xaxis]]
    c(min(x_values, na.rm = TRUE) * 1.1, max(x_values, na.rm

```

```

m = TRUE) * 1.1)
  })

# Reactive expression to prepare selected data
selected_data <- reactive({
  global_artist_count <- floor((input$globalCount / 100)
* 2000)

  global_data <- music_data %>%
    filter(!(Artist %in% korean_artists)) %>%
    arrange(desc(Total_Revenue)) %>%
    slice(1:global_artist_count)

  korean_data <- music_data %>% filter(Artist %in% korean
_artists)

  bind_rows(global_data, korean_data) %>%
    mutate(Artist_Type = ifelse(Artist %in% korean_artist
s, "Korean Artist", "Global Artist"))
  })

output$distributionPlot <- renderPlotly({
  data <- selected_data()

  # Calculate mean values for Korean and Global artists
  korean_mean <- data %>%
    filter(Artist_Type == "Korean Artist") %>%
    summarise(mean_value = mean(.data[[input$xaxis]], na.
rm = TRUE)) %>%
    pull(mean_value)

  global_mean <- data %>%
    filter(Artist_Type == "Global Artist") %>%
    summarise(mean_value = mean(.data[[input$xaxis]], na.
rm = TRUE)) %>%
    pull(mean_value)

  # Create the plot

```

```

p <- ggplot() +
  geom_point(data = data %>% filter(Artist_Type == "Global Artist"),
    aes(x = .data[[input$xaxis]], y = Total_Revenue,
      color = Artist_Type,
      text = paste('Artist:', Artist, '<br>Total Revenue:', Total_Revenue, '<br>', input$xaxis, ':', .data[[input$xaxis]])),
    show.legend = TRUE) +
  geom_point(data = data %>% filter(Artist_Type == "Korean Artist"),
    aes(x = .data[[input$xaxis]], y = Total_Revenue,
      color = Artist_Type,
      text = paste('Artist:', Artist, '<br>Total Revenue:', Total_Revenue, '<br>', input$xaxis, ':', .data[[input$xaxis]])),
    show.legend = TRUE) +
  scale_color_manual(values = c("Korean Artist" = "red", "Global Artist" = "grey")) +
  geom_vline(xintercept = korean_mean, color = "orange", linetype = "dashed", size = 1) +
  geom_vline(xintercept = global_mean, color = "black", linetype = "dashed", size = 1) +
  labs(title = "Distribution of Total Revenue by Selected Feature",
    x = input$xaxis,
    y = "Total Revenue") +
  theme_minimal()

ggplotly(p, tooltip = "text")
})

# GPT Analysis
observeEvent(input$analyze, {
  req(input$api_key)
  data <- selected_data()

```

```

    if(nrow(data) > 0) {
      avg_total_revenue <- round(mean(data$Total_Revenue, na.rm = TRUE), 2)
      avg_selected_feature <- round(mean(data[[input$xaxis]], na.rm = TRUE), 2)

      summary_text <- paste(
        "Overall Data Summary:\n",
        "Average Total Revenue:", avg_total_revenue, "\n",
        "Average", input$xaxis, ":", avg_selected_feature,
        "\n",
        "This data includes both Korean and Global artists."
      )

      prompt <- paste("Provide insights based on the following data summary: Answers must be provided in Korean. Answers should be provided in the following format:
1. Explain what the data you provided is.
2. Provide insights you can find from the data.
3. Provide additional comments based on your analysis of the data.\n", summary_text)

      # GPT API call function
      call_gpt <- function(prompt, api_key) {
        response <- POST(
          url = "https://api.openai.com/v1/chat/completions",
          add_headers(Authorization = paste("Bearer", api_key)),
          content_type("application/json"),
          body = toJSON(list(
            model = "gpt-3.5-turbo",
            messages = list(
              list(role = "user", content = prompt)
            )
          ), auto_unbox = TRUE)
        )
      }
    }
  }

```

```

    )

    # Check API response
    if (response$status_code == 200) {
      result <- content(response, as = "parsed")
      return(result$choices[[1]]$message$content)
    } else {
      return(paste("Error:", response$status_code))
    }
  }

  gpt_answer <- call_gpt(prompt, input$api_key)

  output$gpt_analysis <- renderText({
    paste("Response:", gpt_answer)
  })
} else {
  output$gpt_analysis <- renderText({
    "No data available for analysis."
  })
}
})

# --- Tab 2: Song Radar Chart Comparison ---

selected_song_data <- reactive({
  req(input$selected_song)
  music_data %>%
    filter(Track == input$selected_song)
})

output$artistName <- renderText({
  song_data <- selected_song_data()
  paste(unique(song_data$Artist))
})

# Main Radar Chart for Selected Song
output$mainRadarChart <- renderPlot({

```

```

song_data <- selected_song_data()
vars <- c("Danceability", "Energy", "Key", "Loudness",
"Speechiness")

# Ensure variables are numeric
song_data[vars] <- lapply(song_data[vars], function(x)
as.numeric(as.character(x)))

# Compute min and max for scaling
min_values <- sapply(vars, function(var) min(as.numeric
(music_data[[var]]), na.rm = TRUE))
max_values <- sapply(vars, function(var) max(as.numeric
(music_data[[var]]), na.rm = TRUE))

# Scale the selected song data
song_values <- unlist(song_data[vars])
scaled_song_values <- (song_values - min_values) / (max
_values - min_values)
scaled_song_data <- as.data.frame(t(scaled_song_value
s))
colnames(scaled_song_data) <- vars

# Prepare radar chart data
radar_data <- rbind(rep(1, length(vars)), rep(0, length
(vars)), scaled_song_data)
colnames(radar_data) <- vars

# Plot radar chart
radarchart(radar_data, axistype = 1,
           pcol = "blue", pfc col = alpha("blue", 0.5), p
lwd = 2,
           cglcol = "grey", cglty = 1, axislabcol = "gr
ey", cglwd = 0.8,
           vl cex = 0.8)
legend("topright", legend = input$selected_song, col =
"blue", lty = 1, lwd = 2)
})

```



```

# Radar Chart for Similar Songs
output$similarRadarChart <- renderPlot({
  song_data <- selected_song_data()
  vars <- c("Danceability", "Energy", "Key", "Loudness",
"Speechiness")

  # Ensure variables are numeric
  song_data[vars] <- lapply(song_data[vars], function(x)
as.numeric(as.character(x)))

  # Compute min and max for scaling
  min_values <- sapply(vars, function(var) min(as.numeric
(music_data[[var]]), na.rm = TRUE))
  max_values <- sapply(vars, function(var) max(as.numeric
(music_data[[var]]), na.rm = TRUE))

  # Scale the song vector
  song_vector <- as.numeric(song_data[vars])
  scaled_song_vector <- (song_vector - min_values) / (max
_values - min_values)

  # Prepare all songs data
  all_songs_data <- music_data %>%
  filter(Track != input$selected_song) %>%
  select(Track, Artist, all_of(vars)) %>%
  mutate(across(vars, ~ as.numeric(as.character(.)))) %
>%
  drop_na()

  # Scale the variables for all songs
  for (var in vars) {
    min_val <- min_values[var]
    max_val <- max_values[var]
    all_songs_data[[var]] <- (all_songs_data[[var]] - min
_val) / (max_val - min_val)
  }

  # Compute Euclidean distance

```

```

all_songs_data$distance <- apply(all_songs_data[, vars], 1, function(row) {
  sqrt(sum((row - scaled_song_vector)^2))
})

# Get top 3 similar songs
similar_songs <- all_songs_data %>%
  arrange(distance) %>%
  head(3)

# Prepare radar chart data for similar songs
radar_data <- data.frame(matrix(ncol = length(vars), nrow = 2))
colnames(radar_data) <- vars
radar_data[1, ] <- 1 # Max values
radar_data[2, ] <- 0 # Min values

colors <- c("red", "green", "purple")

for (i in 1:nrow(similar_songs)) {
  # Extract the scaled values for the song
  similar_song_values <- as.numeric(similar_songs[i, vars])

  # Create a data frame with the same column names
  scaled_song_values <- as.data.frame(t(similar_song_values))
  colnames(scaled_song_values) <- vars

  # Bind the data
  radar_data <- rbind(radar_data, scaled_song_values)
}

# Plot radar chart for similar songs
radarchart(radar_data, axistype = 1,
           pcol = colors[1:nrow(similar_songs)],
           pfcol = sapply(colors[1:nrow(similar_songs)], function(col) alpha(col, 0.5)),

```

```

        plwd = 2,
        cglcol = "grey", cglty = 1, axislabcol = "grey", cglwd = 0.8,
        vlcey = 0.8)
    legend("topright", legend = similar_songs$Track, col =
colors[1:nrow(similar_songs)], lty = 1, lwd = 2)
  })

# Table of Top 3 Similar Songs
output$similarSongs <- renderTable({
  song_data <- selected_song_data()
  vars <- c("Danceability", "Energy", "Key", "Loudness",
"Speechiness")

  # Ensure variables are numeric
  song_data[vars] <- lapply(song_data[vars], function(x)
as.numeric(as.character(x)))

  # Compute min and max for scaling
  min_values <- sapply(vars, function(var) min(as.numeric
(music_data[[var]]), na.rm = TRUE))
  max_values <- sapply(vars, function(var) max(as.numeric
(music_data[[var]]), na.rm = TRUE))

  # Scale the song vector
  song_vector <- as.numeric(song_data[vars])
  scaled_song_vector <- (song_vector - min_values) / (max
_values - min_values)

  # Prepare all songs data
  all_songs_data <- music_data %>%
    filter(Track != input$selected_song) %>%
    select(Track, Artist, all_of(vars)) %>%
    mutate(across(vars, ~ as.numeric(as.character(.)))) %
>%
    drop_na()

  # Scale the variables for all songs

```

```

    for (var in vars) {
      min_val <- min_values[var]
      max_val <- max_values[var]
      all_songs_data[[var]] <- (all_songs_data[[var]] - min_val) / (max_val - min_val)
    }

    # Compute Euclidean distance
    all_songs_data$distance <- apply(all_songs_data[, vars], 1, function(row) {
      sqrt(sum((row - scaled_song_vector)^2))
    })

    # Get top 3 similar songs
    similar_songs <- all_songs_data %>%
      arrange(distance) %>%
      head(3) %>%
      select(Track, Artist)
    similar_songs
  })
}

# Run the application
shinyApp(ui = ui, server = server)

```

우리 팀은 유저와의 interaction을 위한 2가지 기능을 구현하였다.

GPT analysis

```

# GPT API 호출 함수
call_gpt <- function(prompt, api_key) {
  response <- POST(
    url = "https://api.openai.com/v1/chat/completions",
    add_headers(Authorization = paste("Bearer", api_key)),
    content_type("application/json"),
    body = toJSON(list(

```

```

model = "gpt-4o-mini-2024-07-18", # "gpt-4"로도 변경
가능
messages = list(
  list(role = "user", content = prompt)
), auto_unbox = TRUE)
)

# API 응답 상태와 내용 확인
print(response$status_code) # 상태 코드 출력
print(content(response, as = "text")) # 응답 내용 출력

# API 응답 확인
if (response$status_code == 200) {
  result <- content(response, as = "parsed")
  return(result$choices[[1]]$message$content)
} else {
  return(paste("Error:", response$status_code))
}
}

```

```

# GPT 분석 요청
observeEvent(input$analyze, {
  req(input$api_key)
  data <- selected_data()

  if(nrow(data) > 0) {
    avg_total_revenue <- round(mean(data$Total_Revenue, n
a.rm = TRUE), 2)
    avg_selected_feature <- round(mean(data[[input$xaxis
s]], na.rm = TRUE), 2)

    summary_text <- paste(
      "Overall Data Summary:\n",
      "Average Total Revenue:", avg_total_revenue, "\n",
      "Average", input$xaxis, ":", avg_selected_feature,

```

```

"\n",
  "This data includes both Korean and Global artists."
)

prompt <- paste("Provide insights based on the following data summary: Answers must be provided in Korean. Answers should be provided in the following format
1. explain what the data you provided is.
2. provide insights you can find from the data.
3. provide additional comments based on your analysis of the data.\n", summary_text)
gpt_answer <- call_gpt(prompt, input$api_key)

output$gpt_analysis <- renderText({
  # GPT 응답과 상태 코드를 같이 출력
  paste("Response:", gpt_answer)
})
} else {
  output$gpt_analysis <- renderText({
    "No data available for analysis."
  })
}
})

```

GPT에게 유저가 보고있는 데이터그래프의 내용을 분석하여 제공받는 코드로 전체 revenue에 대한 특정 feature를 제공받은 뒤 각각에 대한 데이터를 한국과 해외 아티스트에 대한 비교를 수행하고 그 결과를 제공받는다.

Artist Revenue Distribution

Song Radar Chart Comparison

Enter your OpenAI API Key:

Select Feature for X-axis:

Energy

Select Top Percentage of Global Artists:

1/3

Analyze with GPT

Distribution of Total Revenue by Selected Feature

Response: 1. 데이터 요약은 다음과 같습니다: 평균 총 수익은 83432603.47이며, 평균 에너지는 0.76입니다. 한국 및 글로벌 아티스트들 모두 포함한 데이터입니다. 2. 이 데이터에서 찾을 수 있는 인사이트는 다음과 같습니다: - 전반적으로, 아티스트들의 평균 수익은 83432603.47로 나타났습니다. 이는 상당히 높은 숫자이며, 음악 산업이 큰 돈을 벌다는 것을 시사합니다. - 아티스트들의 음악이 평균적으로 0.76의 에너지를 갖는다는 것은, 고강도의 활기찬 음악을 만들거나 선호한다는 것을 암시할 수 있습니다. 3. 추가로 분석한 측면에서, 이 데이터에서는 한국과 글로벌 아티스트들의 수익과 에너지에 대한 평균값을 비교해 볼 수 있을 것입니다. 한국 아티스트들이 어떤 특징을 갖고 있는지, 그리고 글로벌 아티스트들과 비교했을 때 어떤 차별점이 있는지 등을 살펴볼 수 있을 것입니다. 이를 통해 음악 시장의 트렌드와 아티스트들의 특성을 더 잘 이해할 수 있을 것입니다.

song recoment

```

    mainPanel(
      h3("Main Radar Chart (Selected Song)"),
      plotOutput("mainRadarChart"),
      h3("Radar Chart (Top 3 Similar Songs)"),
      plotOutput("similarRadarChart"),
      h4("Artist:"),
      textOutput("artistName"),
      h3("Top 3 Similar Songs:"),
      tableOutput("similarSongs")
    )
  )
}

# --- Tab 2: Song Radar Chart Comparison ---

selected_song_data <- reactive({
  req(input$selected_song)
  music_data %>%

```

```

    filter(Track == input$selected_song)
  })

output$artistName <- renderText({
  song_data <- selected_song_data()
  paste(unique(song_data$Artist))
})

# Main Radar Chart for Selected Song
output$mainRadarChart <- renderPlot({
  song_data <- selected_song_data()
  vars <- c("Danceability", "Energy", "Key", "Loudness",
"Speechiness")

  # Ensure variables are numeric
  song_data[vars] <- lapply(song_data[vars], function(x)
as.numeric(as.character(x)))

  # Compute min and max for scaling
  min_values <- sapply(vars, function(var) min(as.numeric
(music_data[[var]]), na.rm = TRUE))
  max_values <- sapply(vars, function(var) max(as.numeric
(music_data[[var]]), na.rm = TRUE))

  # Scale the selected song data
  song_values <- unlist(song_data[vars])
  scaled_song_values <- (song_values - min_values) / (max
_values - min_values)
  scaled_song_data <- as.data.frame(t(scaled_song_value
s))
  colnames(scaled_song_data) <- vars

  # Prepare radar chart data
  radar_data <- rbind(rep(1, length(vars)), rep(0, length
(vars)), scaled_song_data)
  colnames(radar_data) <- vars

  # Plot radar chart

```



```

    radarchart(radar_data, axistype = 1,
               pcol = "blue", pfc col = alpha("blue", 0.5), p
lwd = 2,
               cglcol = "grey", cglty = 1, axislabcol = "gr
ey", cglwd = 0.8,
               vlce x = 0.8)
    legend("topright", legend = input$selected_song, col =
"blue", lty = 1, lwd = 2)
  })

# Radar Chart for Similar Songs
output$similarRadarChart <- renderPlot({
  song_data <- selected_song_data()
  vars <- c("Danceability", "Energy", "Key", "Loudness",
"Speechiness")

  # Ensure variables are numeric
  song_data[vars] <- lapply(song_data[vars], function(x)
as.numeric(as.character(x)))

  # Compute min and max for scaling
  min_values <- sapply(vars, function(var) min(as.numeric
(music_data[[var]]), na.rm = TRUE))
  max_values <- sapply(vars, function(var) max(as.numeric
(music_data[[var]]), na.rm = TRUE))

  # Scale the song vector
  song_vector <- as.numeric(song_data[vars])
  scaled_song_vector <- (song_vector - min_values) / (max
_values - min_values)

  # Prepare all songs data
  all_songs_data <- music_data %>%
    filter(Track != input$selected_song) %>%
    select(Track, Artist, all_of(vars)) %>%
    mutate(across(vars, ~ as.numeric(as.character(.)))) %
>%
    drop_na()

```

```

# Scale the variables for all songs
for (var in vars) {
  min_val <- min_values[var]
  max_val <- max_values[var]
  all_songs_data[[var]] <- (all_songs_data[[var]] - min_val) / (max_val - min_val)
}

# Compute Euclidean distance
all_songs_data$distance <- apply(all_songs_data[, vars], 1, function(row) {
  sqrt(sum((row - scaled_song_vector)^2))
})

# Get top 3 similar songs
similar_songs <- all_songs_data %>%
  arrange(distance) %>%
  head(3)

# Prepare radar chart data for similar songs
radar_data <- data.frame(matrix(ncol = length(vars), nrow = 2))
colnames(radar_data) <- vars
radar_data[1, ] <- 1 # Max values
radar_data[2, ] <- 0 # Min values

colors <- c("red", "green", "purple")

for (i in 1:nrow(similar_songs)) {
  # Extract the scaled values for the song
  similar_song_values <- as.numeric(similar_songs[i, vars])

  # Create a data frame with the same column names
  scaled_song_values <- as.data.frame(t(similar_song_values))
  colnames(scaled_song_values) <- vars

```

```

# Bind the data
radar_data <- rbind(radar_data, scaled_song_values)
}

# Plot radar chart for similar songs
radarchart(radar_data, axistype = 1,
           pcol = colors[1:nrow(similar_songs)],
           pfc col = sapply(colors[1:nrow(similar_songs)], function(col) alpha(col, 0.5)),
           plwd = 2,
           cglcol = "grey", cglty = 1, axislabcol = "grey", cglwd = 0.8,
           vlce x = 0.8)
legend("topright", legend = similar_songs$Track, col = colors[1:nrow(similar_songs)], lty = 1, lwd = 2)
})

# Table of Top 3 Similar Songs
output$similarSongs <- renderTable({
  song_data <- selected_song_data()
  vars <- c("Danceability", "Energy", "Key", "Loudness", "Speechiness")

  # Ensure variables are numeric
  song_data[vars] <- lapply(song_data[vars], function(x) as.numeric(as.character(x)))

  # Compute min and max for scaling
  min_values <- sapply(vars, function(var) min(as.numeric(music_data[[var]]), na.rm = TRUE))
  max_values <- sapply(vars, function(var) max(as.numeric(music_data[[var]]), na.rm = TRUE))

  # Scale the song vector
  song_vector <- as.numeric(song_data[vars])
  scaled_song_vector <- (song_vector - min_values) / (max_values - min_values)

```

```

# Prepare all songs data
all_songs_data <- music_data %>%
  filter(Track != input$selected_song) %>%
  select(Track, Artist, all_of(vars)) %>%
  mutate(across(vars, ~ as.numeric(as.character(.)))) %
>%
  drop_na()

# Scale the variables for all songs
for (var in vars) {
  min_val <- min_values[var]
  max_val <- max_values[var]
  all_songs_data[[var]] <- (all_songs_data[[var]] - min
_val) / (max_val - min_val)
}

# Compute Euclidean distance
all_songs_data$distance <- apply(all_songs_data[, var
s], 1, function(row) {
  sqrt(sum((row - scaled_song_vector)^2))
})

# Get top 3 similar songs
similar_songs <- all_songs_data %>%
  arrange(distance) %>%
  head(3) %>%
  select(Track, Artist)
similar_songs
})

```

Music Data Analysis Dashboard

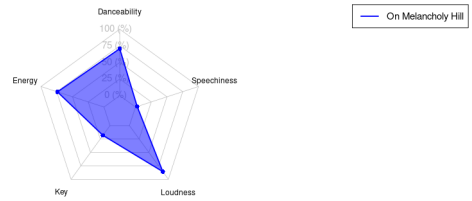
Artist Revenue Distribution

Song Radar Chart Comparison

Select a song:

On Melancholy Hill

Main Radar Chart (Selected Song)



Radar Chart (Top 3 Similar Songs)



song을 선택하면 이와 관련하여 비슷한 3개의 곡을 선택하여 결과를 비교한다.

퍼블리시 : <https://alfinwilliam.shinyapps.io/practice8/>